Documento de Especificación de Requerimientos: TGR (True Gamers Reviews)

Integrantes: Belle-Isle, Nathan · Guixeras Marrufo, Salim · Olvera Aragón, Yusef Alejandro

Docente: Soberanis Várguez, Manuel Jesús

Fecha: 5 de noviembre de 2026

Tabla de contenido

Objetivo de la aplicación....2
Stack de desarrollo propuesto....2
Arquitectura del sistema....4
Flujos principales del usuario....8
Wireframes....11
Sistema de diseño....23

Objetivo de la aplicación

TGR (True Gamers Reviews) es una plataforma web diseñada para la publicación, gestión y consulta de reseñas de videojuegos, tanto para los jugadores como para el personal editorial y administrativo. Su objetivo es que cualquier persona encuentre un juego y lea su reseña en menos de tres clics, comparando en una sola vista tres opiniones independientes: la calificación oficial de TGR, el promedio de revistas especializadas (convertido a 5 estrellas) y el promedio de la comunidad.
El sistema tiene cuatro roles: visitante (consulta el catálogo y las reseñas), usuario registrado (califica de 1 a 5 estrellas, comenta y guarda favoritos), editor/reseñador (publica reseñas oficiales de TGR o de su revista afiliada) y administrador (mantiene el catálogo, gestiona cuentas y roles, y modera comentarios).

Stack de desarrollo propuesto

La aplicación es una app web full stack de JavaScript: SvelteKit genera las páginas en el servidor y Svelte agrega la interactividad en el navegador. Los formularios funcionan aun sin JavaScript. La base de datos es SQLite en un solo archivo, sin servidor aparte. Las pruebas usan Vitest y Playwright, y el diseño se hizo en Figma.
Repositorio en GitHub: https://github.com/SangonapouSchool/tgr-true-gamers-reviews







Componente	Nombre	Uso	Doc
Frontend	HTML 5, CSS 3	Presentación y recepción de datos del usuario	https://developer.mozilla.org/es/docs/Web/HTML https://developer.mozilla.org/es/docs/Web/CSS
	Tailwind CSS 4	Librería CSS; los tokens del sistema de diseño se definen en @theme	https://tailwindcss.com/
	Svelte 5	Framework de componentes JavaScript (runes $state, $derived y $props)	https://svelte.dev/
Backend	SvelteKit 3 sobre Node.js 22+	Rutas, renderizado en servidor, funciones load y acciones de formulario	https://svelte.dev/docs/kit

Base de datos	SQLite (módulo node:sqlite)	Base de datos relacional basada en archivos	https://sqlite.org/
API Externa	Ninguna	Ninguna	
Pruebas	Vitest 5 y Playwright	Pruebas unitarias, de integración y funcionales de extremo a extremo en el navegador	https://vitest.dev/
Diseño	Figma	Sistema de diseño, componentes, wireframes y flujos	https://www.figma.com/design/mObYzvsQU5N0H4yPkS4IAR
Seguridad	node:crypto (scrypt)	Cifrado de contraseñas con sal aleatoria y sesiones en cookie httpOnly	https://nodejs.org/api/crypto.html

Arquitectura del sistema

La aplicación sigue una arquitectura en capas sobre SvelteKit. El navegador recibe páginas ya generadas en el servidor y Svelte agrega la interactividad (mejora progresiva). Toda la lógica de negocio vive en el servidor, en el módulo repo.js, y los datos se guardan en SQLite.

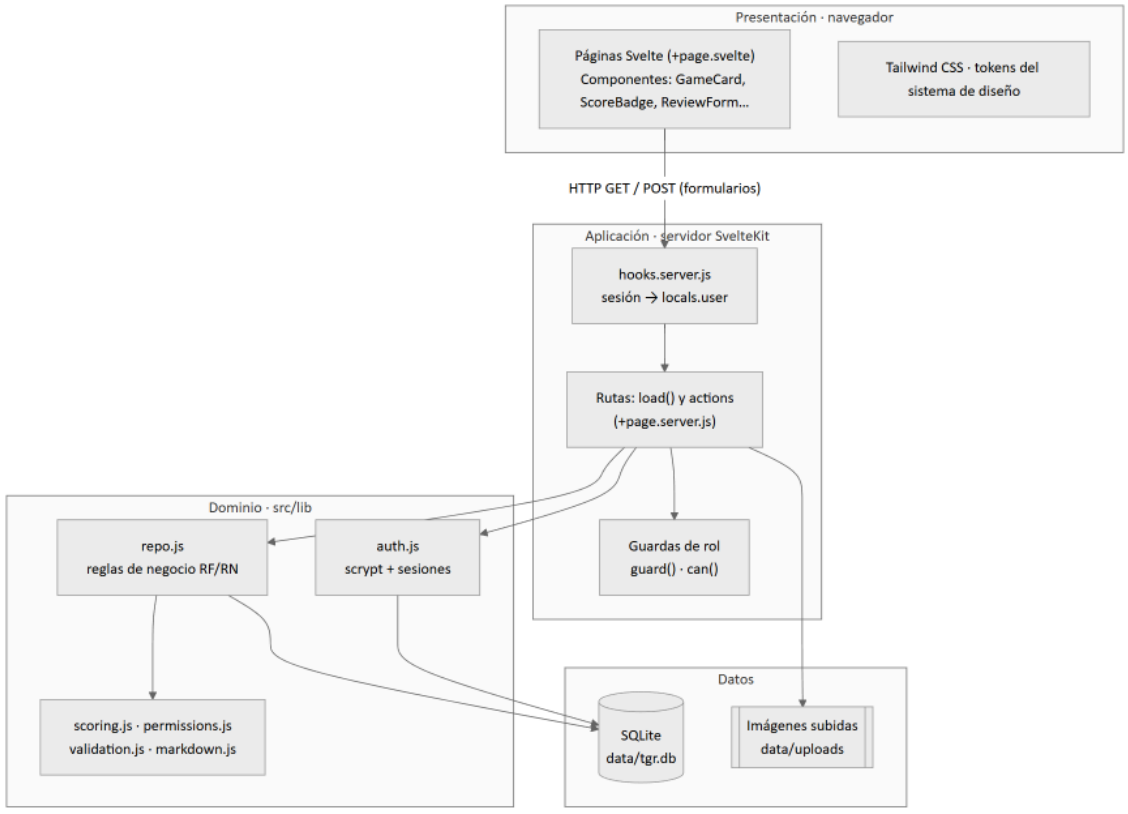


Diagrama 1. Arquitectura en capas: presentación (navegador), aplicación (servidor SvelteKit), dominio (reglas de negocio) y datos (SQLite e imágenes subidas).

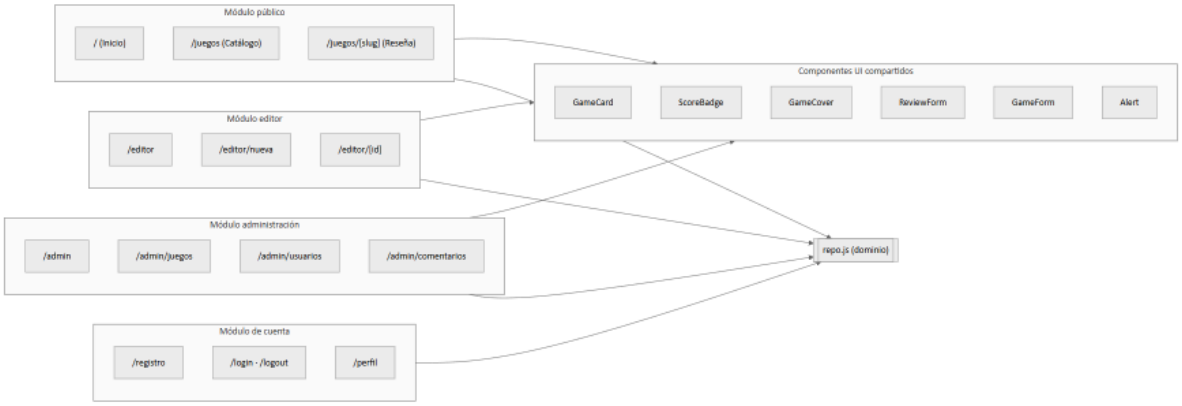


Diagrama 2. Componentes: módulos público, de cuenta, de editor y de administración, que comparten los componentes de interfaz y el repositorio de datos.

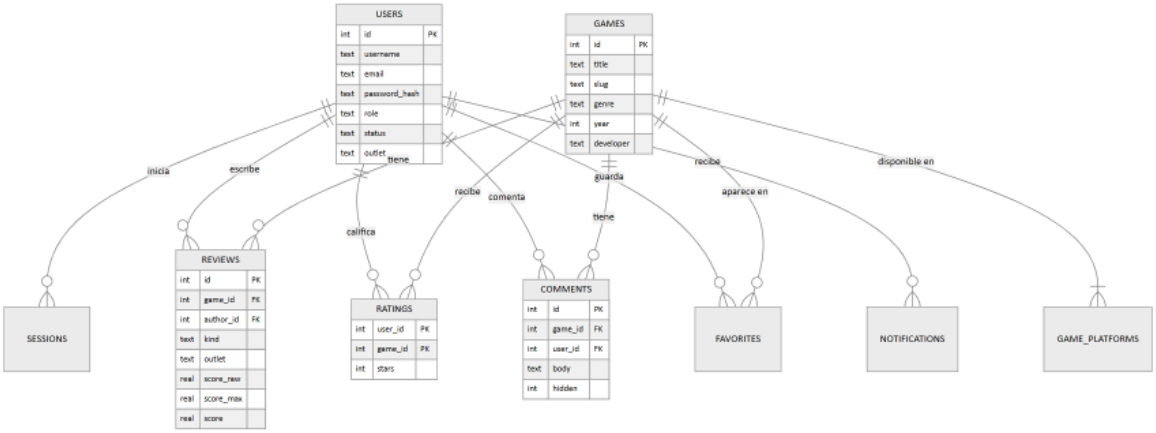


Diagrama 3. Modelo de datos: usuarios, videojuegos y sus plataformas, reseñas, calificaciones, comentarios, favoritos y notificaciones.

Reglas en la base de datos: el índice único one_official_per_game impide una segunda reseña oficial por juego (RN6), la llave compuesta ratings(user_id, game_id) permite una sola calificación por

usuario y juego (RN2), y al borrar un juego se borran en cascada sus reseñas, calificaciones y comentarios.

Estructura del código

```
tgr/
├── src/
│   ├── app.css                tokens del sistema de diseño (Tailwind @theme)
│   ├── hooks.server.js        carga la sesión en cada petición
│   ├── lib/
│   │   ├── scoring.js         RN1, RN3, RN5 (conversión y promedios)
│   │   ├── permissions.js     permisos por rol (RN7)
│   │   ├── validation.js      validaciones (RNF5)
│   │   ├── markdown.js        formato seguro de reseñas (RF5)
│   │   ├── components/        GameCard, ScoreBadge, ReviewForm, …
│   │   └── server/            db.js, auth.js, repo.js, uploads.js
│   └── routes/                páginas (público, cuenta, editor, admin)
├── scripts/                   seed.js (datos demo), e2e-evidencia.js
└── tests/                     unit.test.js, integration.test.js
```

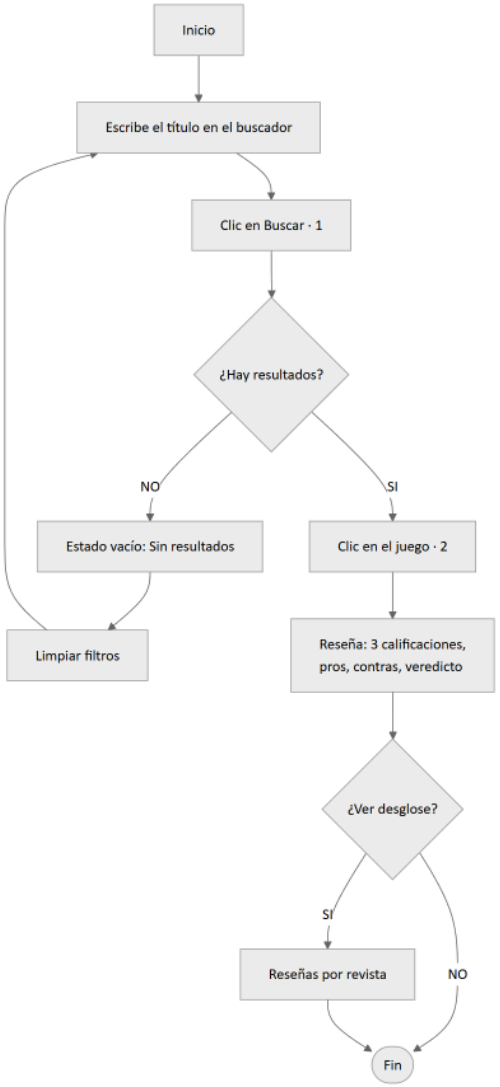
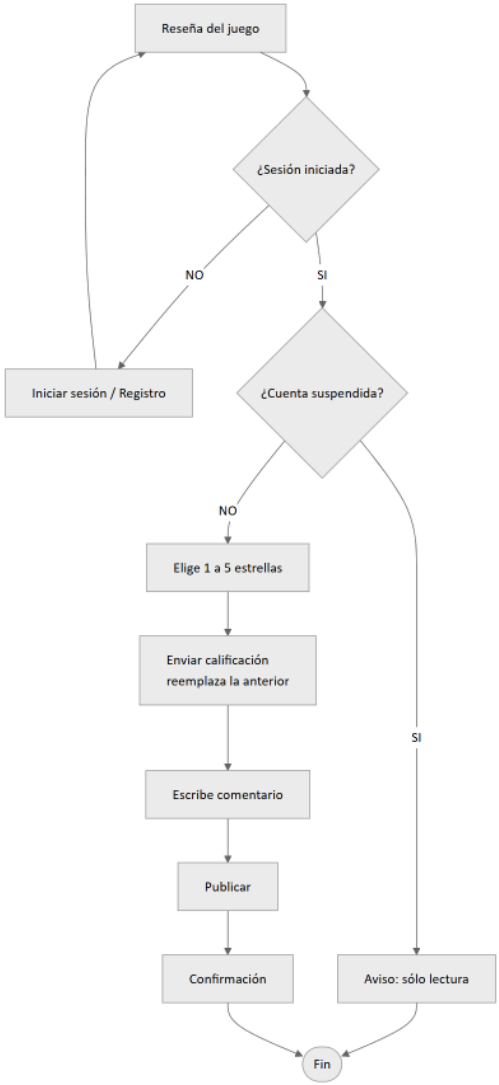
Seguridad y control de acceso

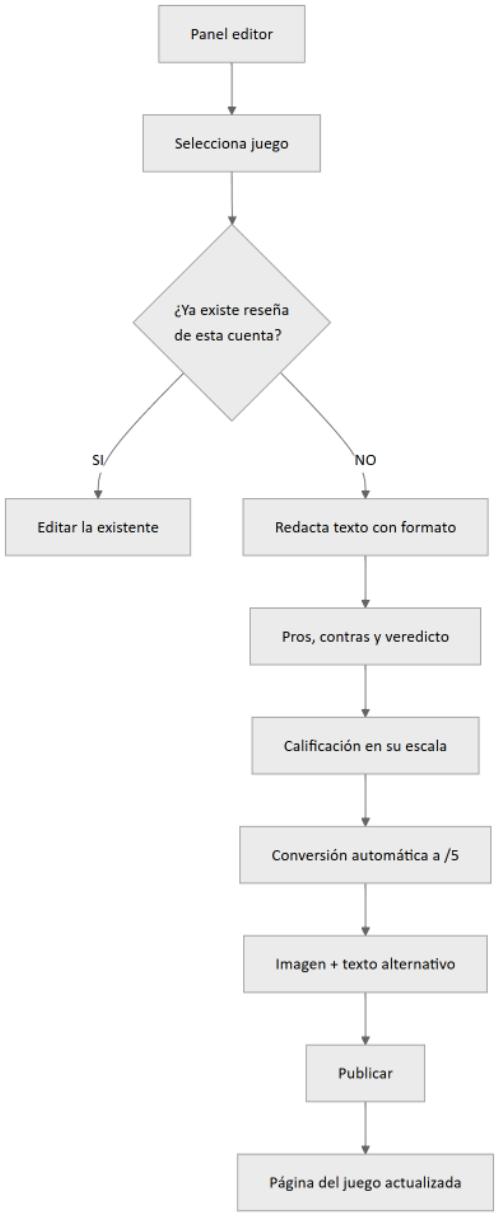
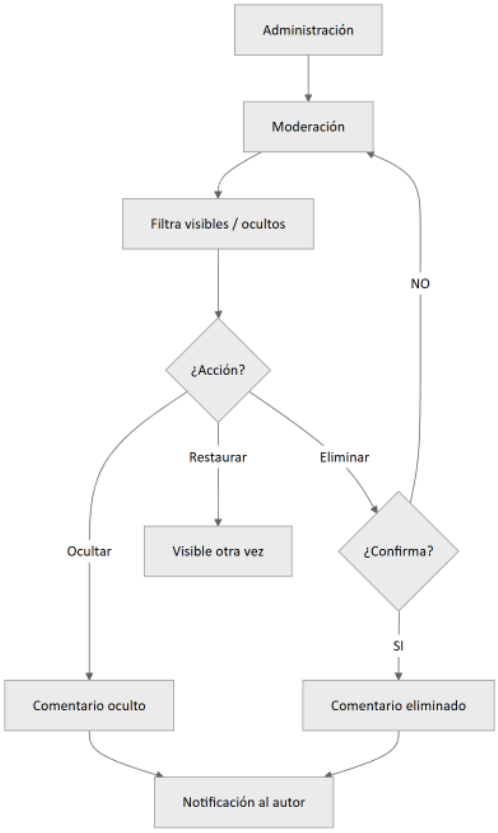
Riesgo	Medida
Contraseñas expuestas	Cifrado con scrypt y sal aleatoria de 16 bytes; la comparación se hace en tiempo constante (RNF5).
Robo de sesión	Token aleatorio de 256 bits en cookie httpOnly, SameSite=Lax y Secure en producción; la sesión expira a los 30 días.
Acceso no autorizado	Guardas en las secciones /editor y /admin, y verificación de permisos en cada operación del repositorio.
Inyección SQL	Todas las consultas usan sentencias preparadas con parámetros.
XSS en reseñas	El formato de reseña escapa todo el HTML antes de convertir títulos, negritas y listas.
CSRF	SvelteKit verifica el origen de los formularios POST.
Subida de archivos	Sólo PNG, JPG, WEBP o GIF de hasta 3 MB, con nombre aleatorio y servidos por una ruta que valida el nombre.

Flujos principales del usuario

Los flujos describen cómo cada rol completa sus tareas principales. Los diagramas se hicieron con Mermaid y su versión visual también está en la página 03 · Flujos del archivo de Figma. El flujo de cliente 1 cumple el criterio de llegar a una reseña en máximo 3 clics (RNF6).

Se documentan dos flujos de cliente final y dos de backoffice:

Flujo cliente 1: visitante busca y lee una reseña	Flujo cliente 2: usuario califica y comenta
	

Flujo backoffice 1: editor publica una reseña	Flujo backoffice 2: administrador modera un comentario
	

Nota:
Los diagramas de arriba fueron realizados con https://mermaid.live/

Este es el código fuente:

Flujo cliente 1: visitante busca y lee una reseña

```
flowchart TD
  Inicio --> Buscar[Escribe el título en el buscador]
  Buscar --> Clic1[Clic en Buscar] --> R{¿Hay resultados?}
  R -->|NO| Vacio[Sin resultados] --> Buscar
  R -->|SI| Clic2[Clic en el juego] --> Resena[Reseña: 3 calificaciones]
  Resena --> D{¿Ver desglose?}
  D -->|SI| Desglose[Reseñas por revista] --> Fin([Fin])
  D -->|NO| Fin
```

Flujo cliente 2: usuario califica y comenta

```
flowchart TD
  A[Reseña del juego] --> S{¿Sesión iniciada?}
  S -->|NO| L[Iniciar sesión / Registro] --> A
  S -->|SI| Su{¿Cuenta suspendida?}
  Su -->|SI| RO[Aviso: sólo lectura] --> Fin([Fin])
  Su -->|NO| E[Elige 1 a 5 estrellas] --> Env[Enviar calificación]
  Env --> C[Escribe comentario] --> P[Publicar] --> Fin
```

Flujo backoffice 1: editor publica una reseña

```
flowchart TD
  P[Panel editor] --> J[Selecciona juego]
  J --> X{¿Ya existe reseña de esta cuenta?}
  X -->|SI| Ed[Editar la existente]
  X -->|NO| T[Redacta texto con formato]
  T --> PC[Pros, contras y veredicto] --> Cal[Calificación en su escala]
  Cal --> Conv[Conversión automática a /5] --> Img[Imagen + texto alternativo]
  Img --> Pub[Publicar] --> Pag[Página del juego]
```

Flujo backoffice 2: administrador modera un comentario

```
flowchart TD
  A[Administración] --> M[Moderación] --> F[Filtra visibles / ocultos]
  F --> D{¿Acción?}
  D -->|Ocultar| H[Comentario oculto] --> N[Notificación al autor]
  D -->|Eliminar| Conf{¿Confirma?}
  Conf -->|NO| M
  Conf -->|SI| Del[Comentario eliminado] --> N
  D -->|Restaurar| V[Visible otra vez]
```

Wireframes

Prototipo en Figma: https://www.figma.com/design/mObYzvsQU5N0H4yPkS4IAR

El archivo de Figma tiene cuatro páginas: 01 · Sistema de diseño (variables de color, estilos de texto y 8 componentes: Button, Input, Chip, ScoreBadge, ScoreChip, GameCover, GameCard y NavBar), 02 · Wireframes (7 pantallas: Inicio, Catálogo, Reseña de juego, Iniciar sesión, Reseña en móvil, Panel editor y Administración de usuarios), 03 · Flujos y 04 · Capturas de la app en ejecución. Las pantallas se construyeron con los componentes del sistema de diseño, así que un cambio en un componente se refleja en todas.

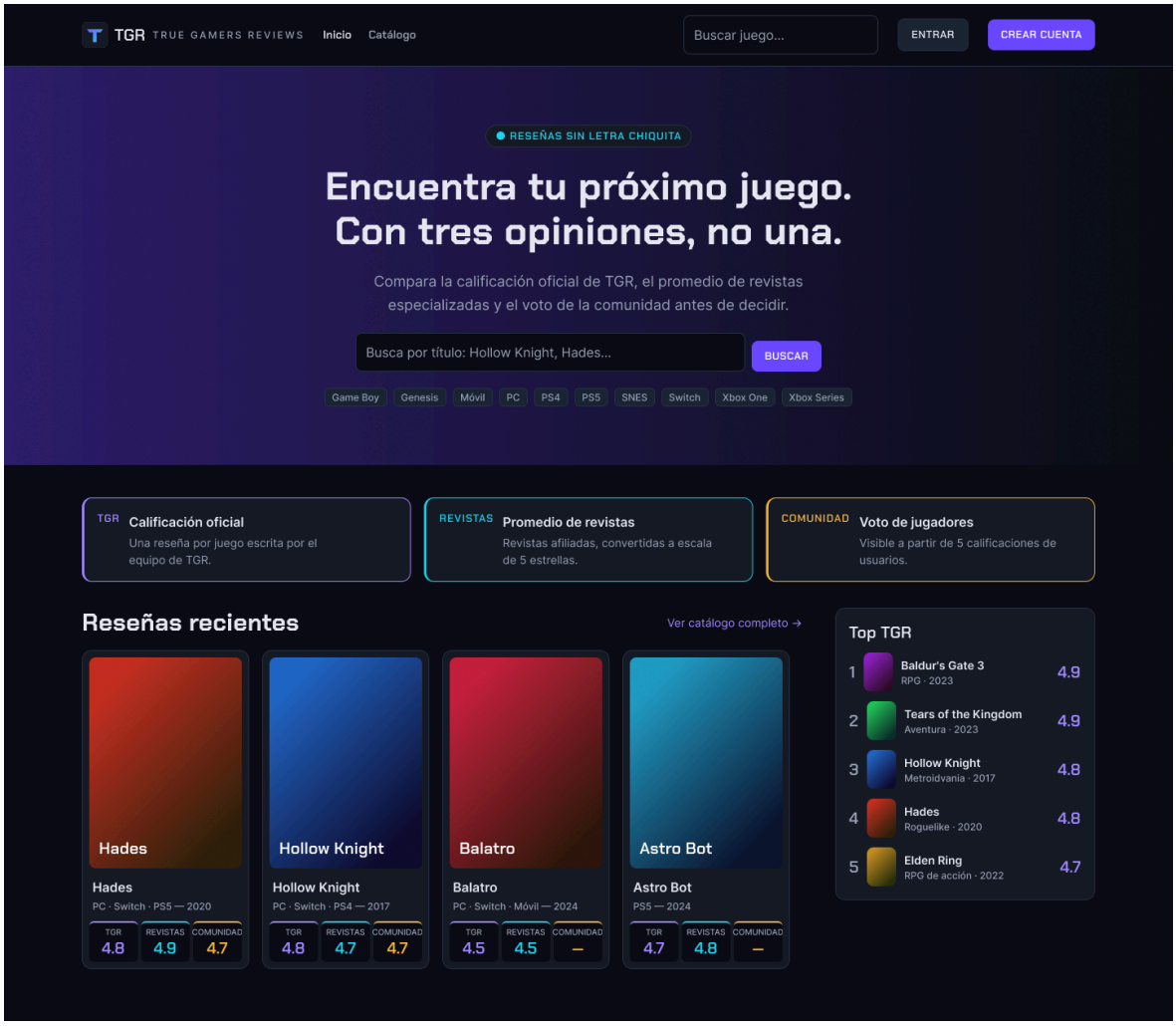


Figura 1. Inicio (visitante): buscador, leyenda de las tres calificaciones, reseñas recientes y Top TGR.

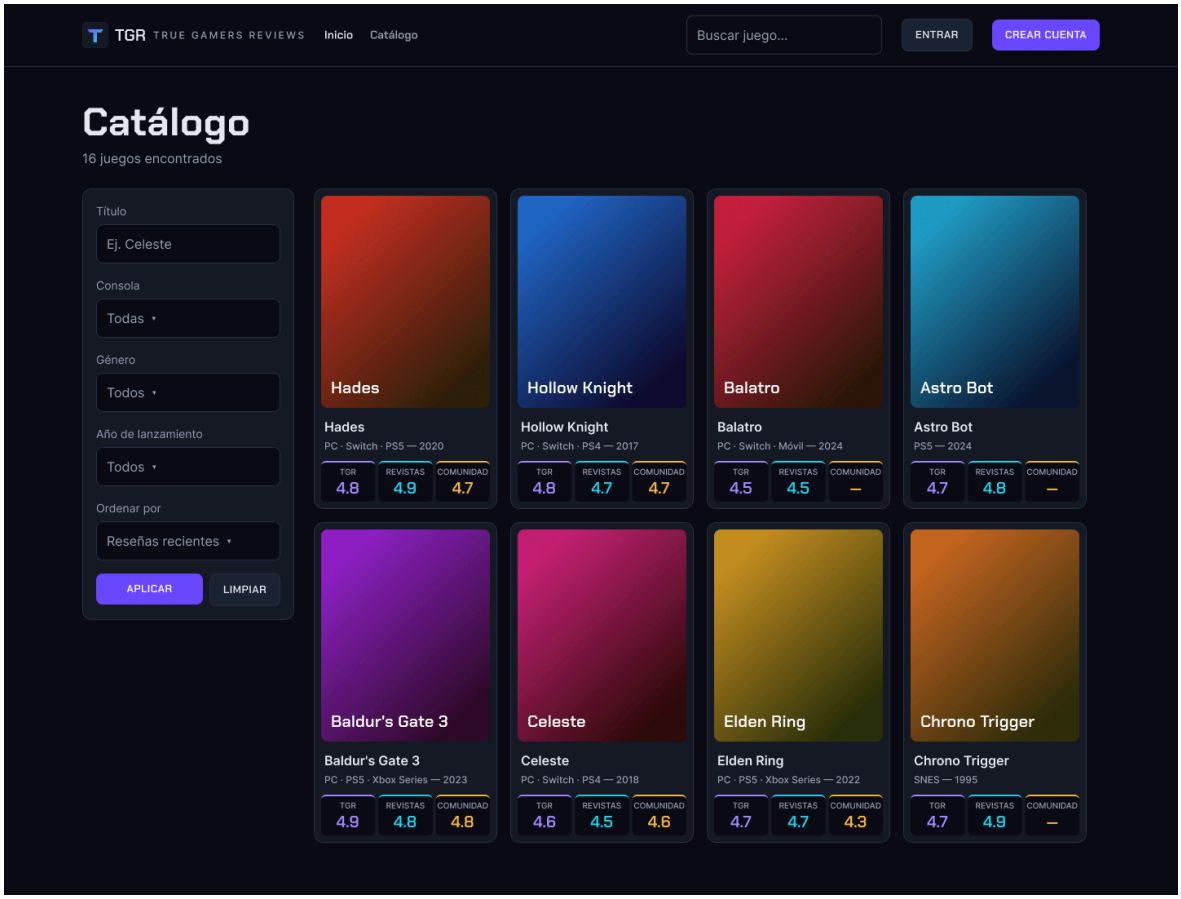


Figura 2. Catálogo: filtros por título, consola, género y año, con tarjetas que muestran las tres calificaciones.

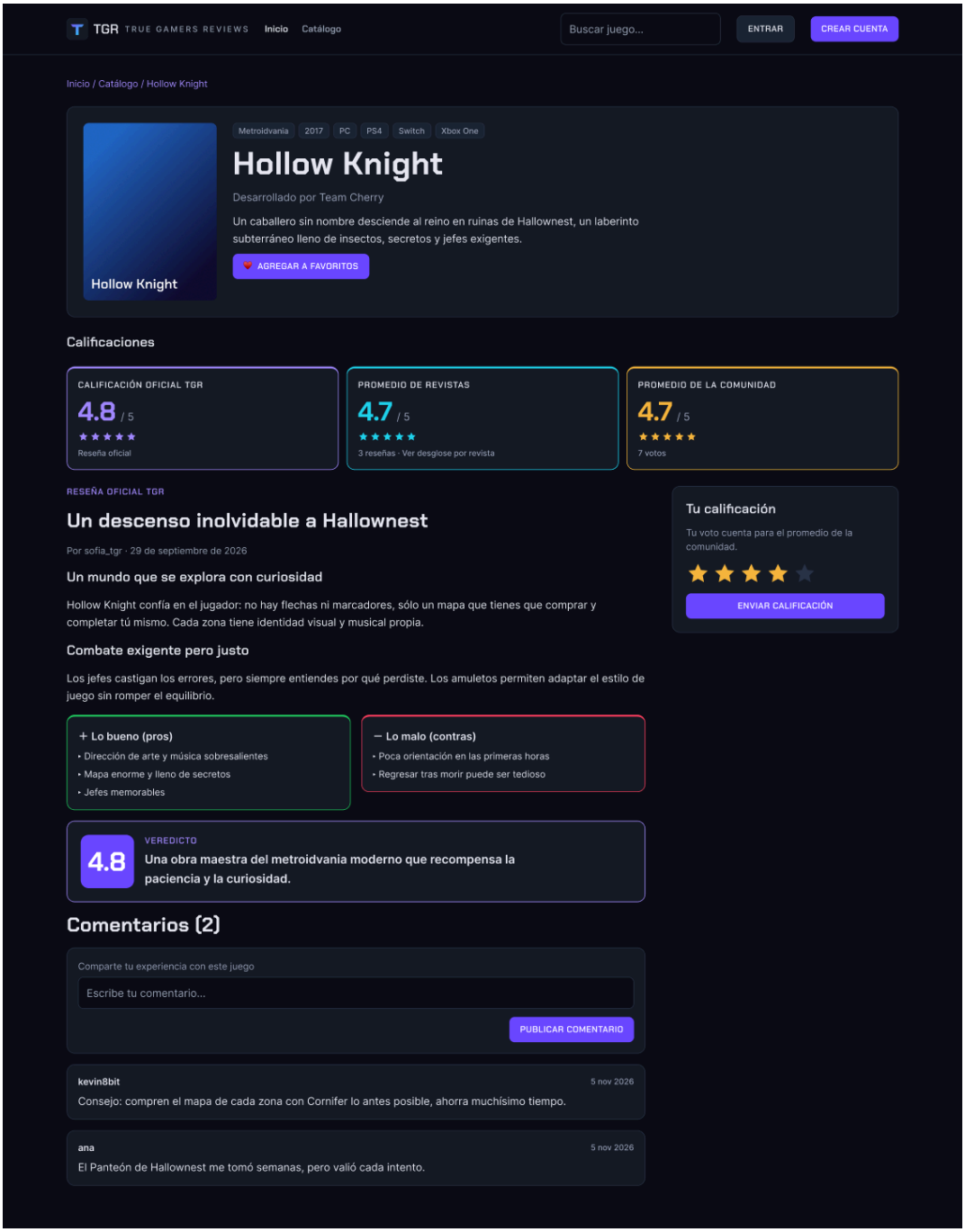


Figura 3. Reseña de juego (usuario registrado): tres calificaciones, reseña oficial, pros y contras, veredicto, calificar y comentar.

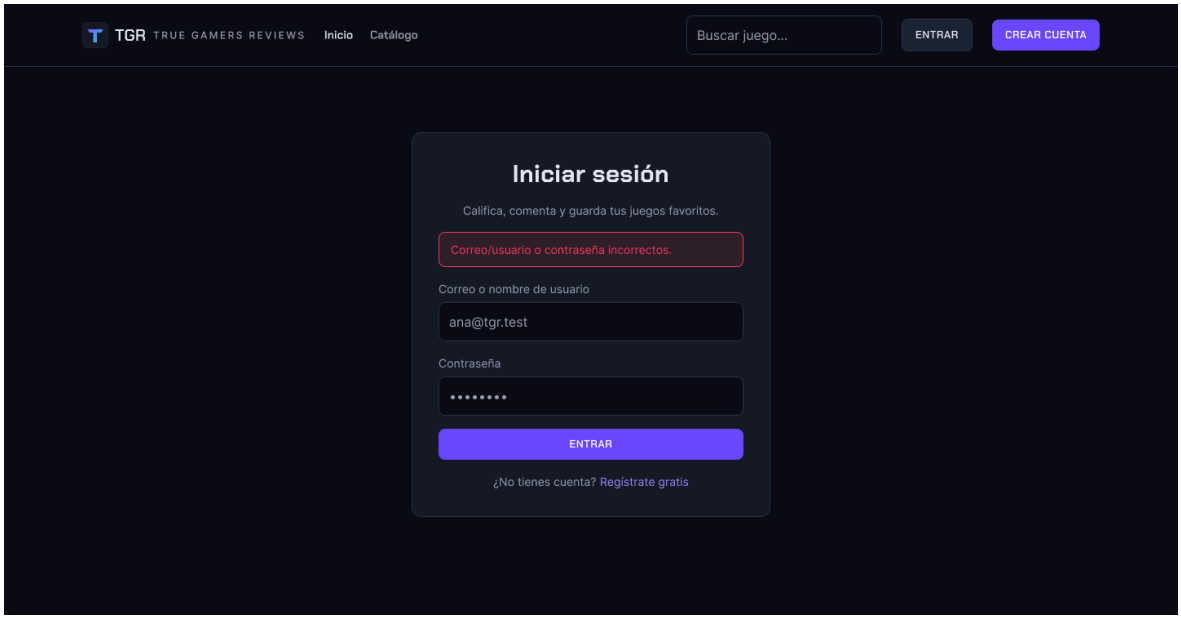


Figura 4. Iniciar sesión con mensaje de error.



Figura 5. Reseña en celular (390 px).

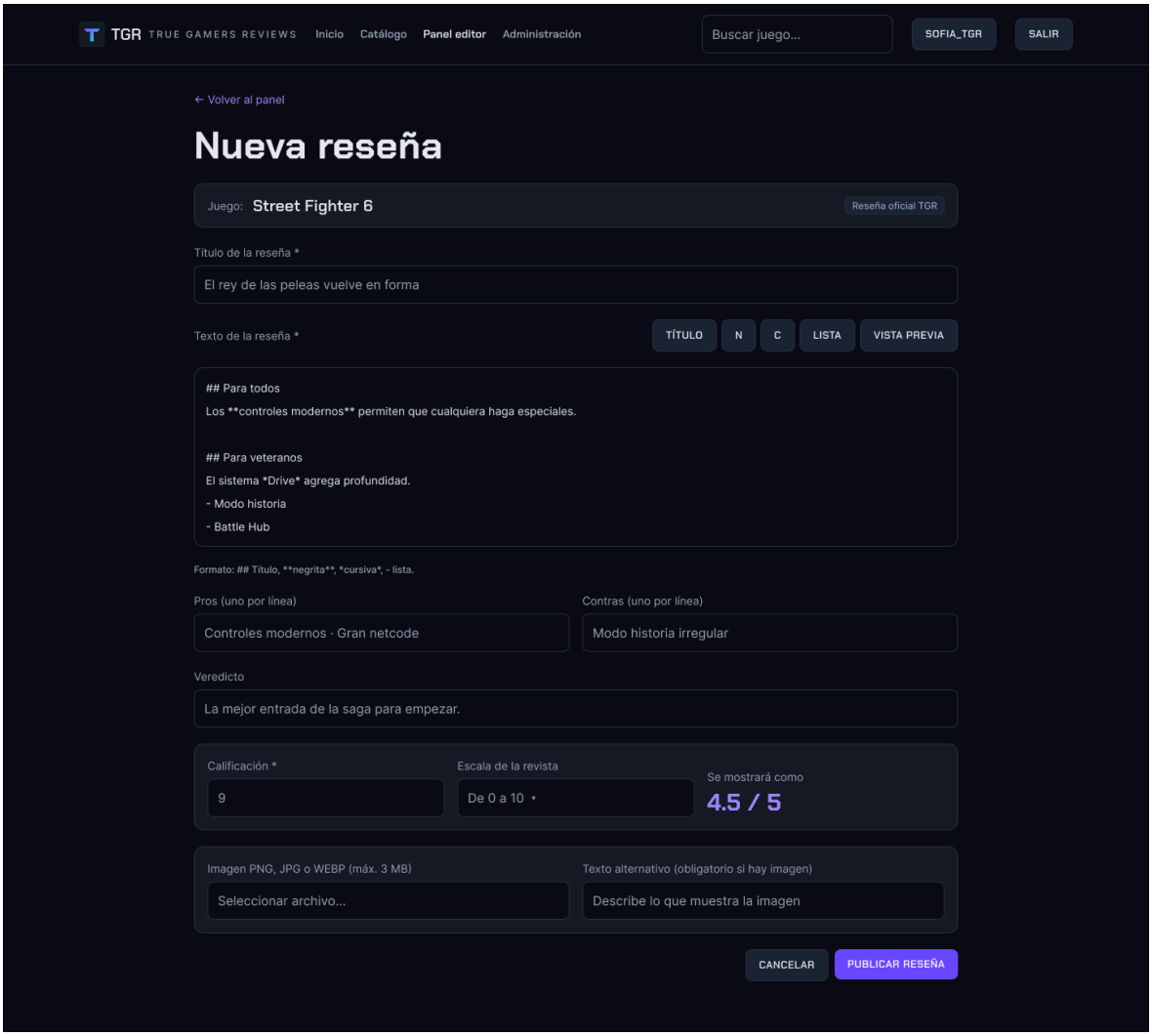


Figura 6. Panel editor, nueva reseña: barra de formato, conversión de escala e imagen con texto alternativo.

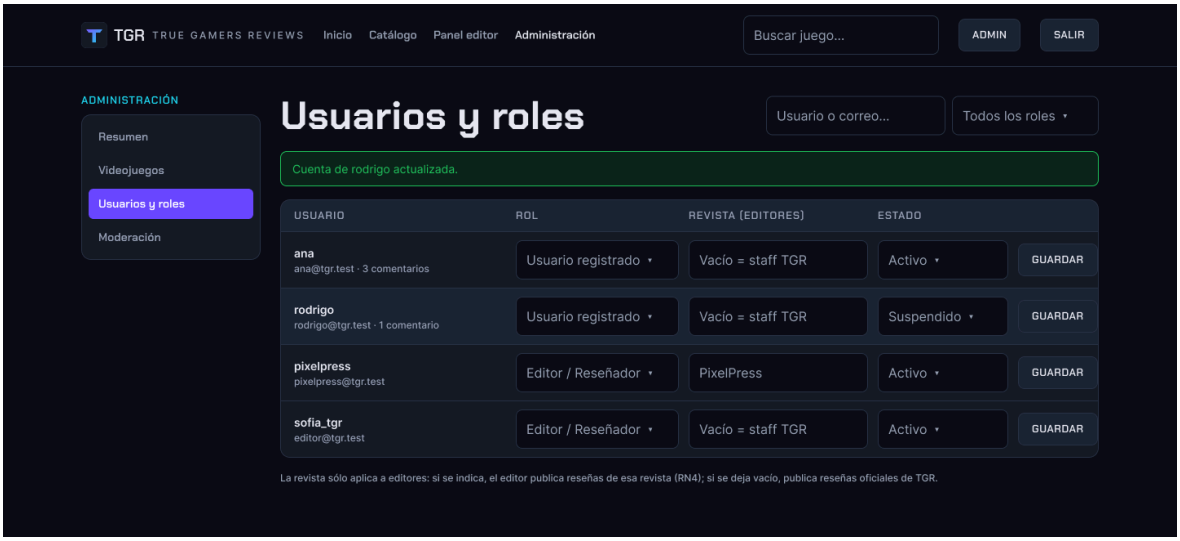


Figura 7. Administración de usuarios y roles: cambio de rol, revista asignada y suspensión.

Capturas de la aplicación en ejecución

Las siguientes capturas son de la aplicación funcionando (npm run dev). Se tomaron automáticamente durante las pruebas de extremo a extremo y muestran el resultado final de los wireframes, ya con las portadas reales de los juegos.

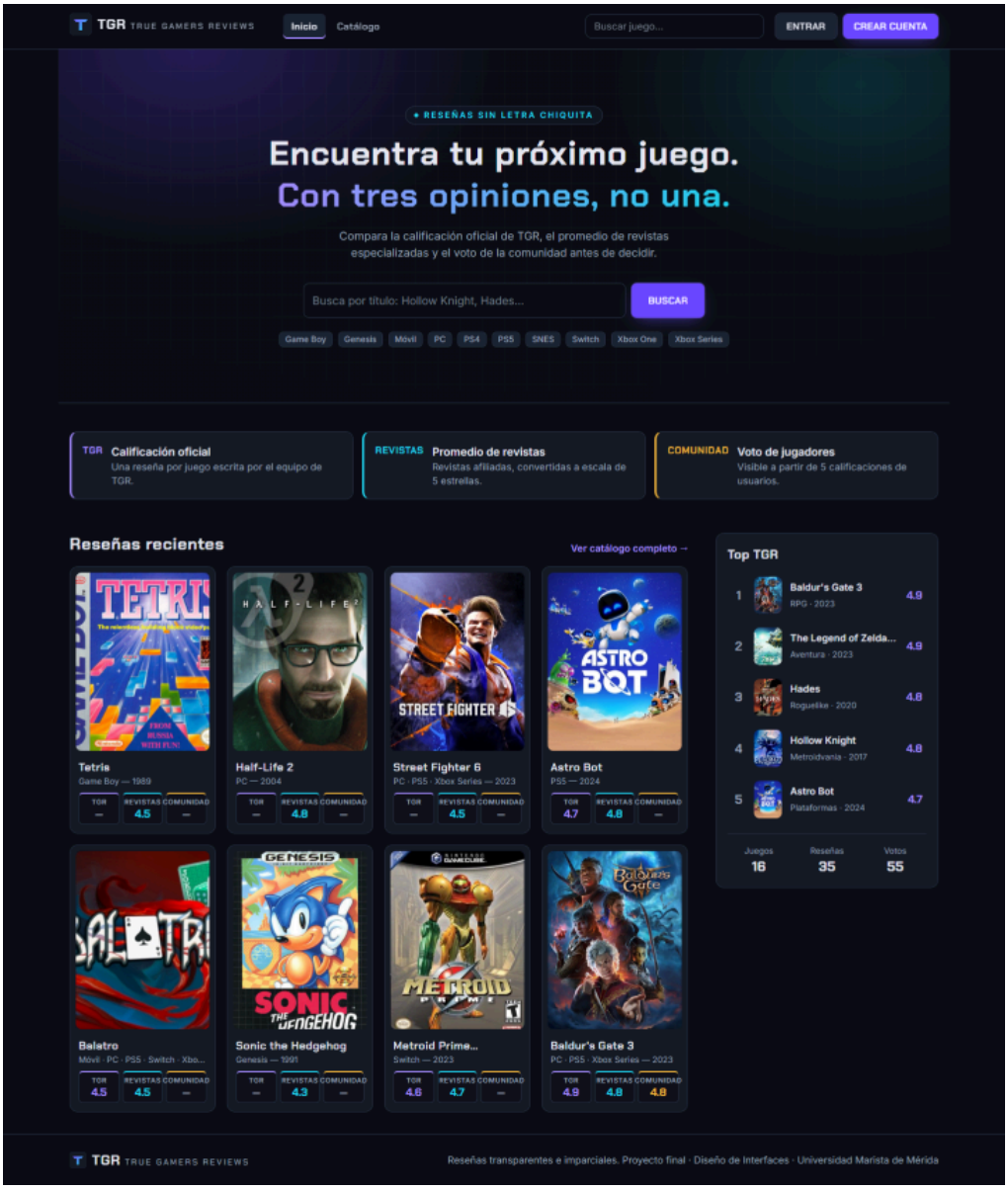


Figura 8. Inicio de la aplicación en ejecución, con las portadas reales de los juegos.

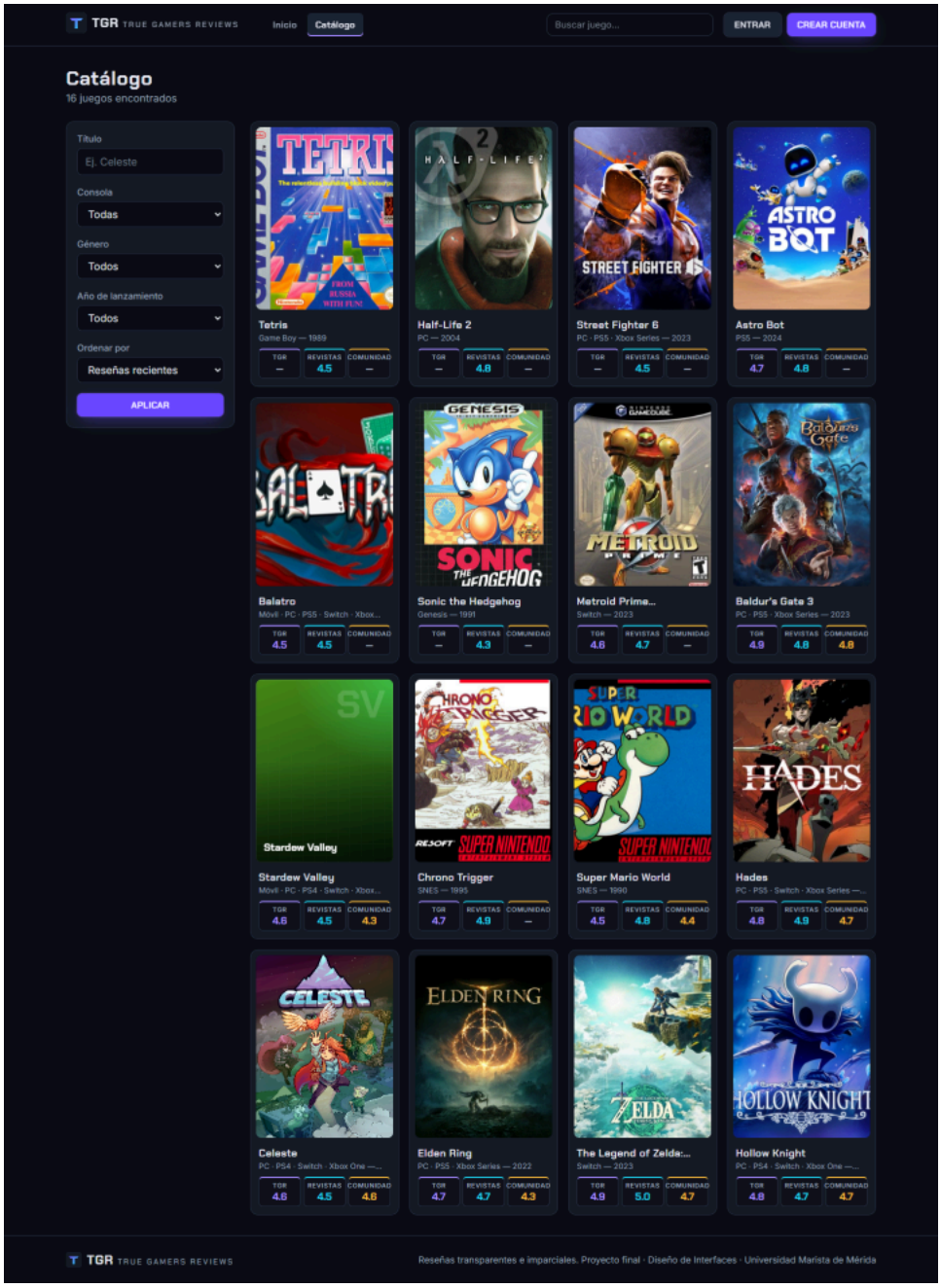


Figura 9. Catálogo con filtros y las tres calificaciones de cada juego.

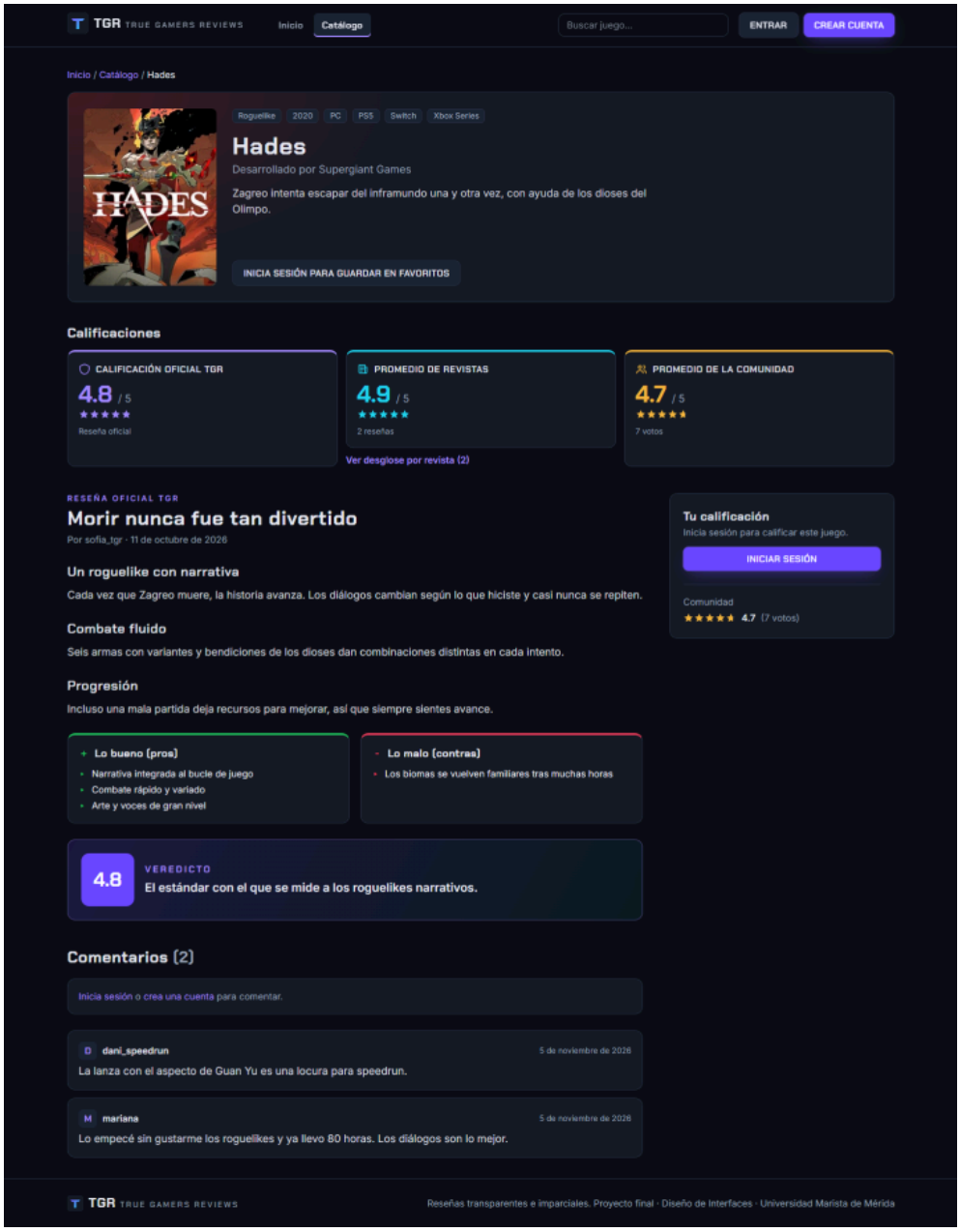


Figura 10. Reseña de Hades: calificaciones, reseña oficial, pros y contras, veredicto y comentarios.

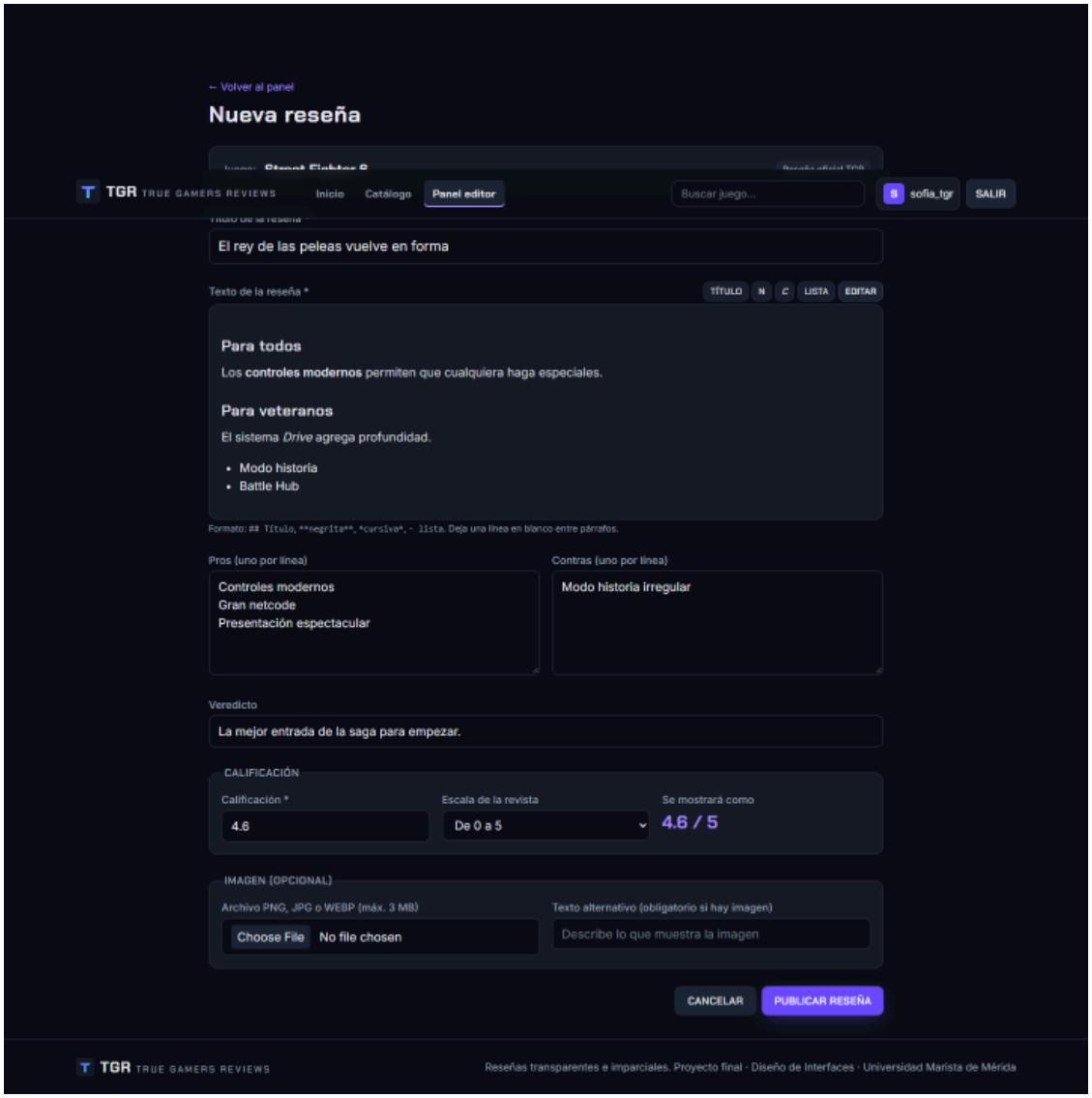


Figura 11. Panel editor: nueva reseña con vista previa del formato.

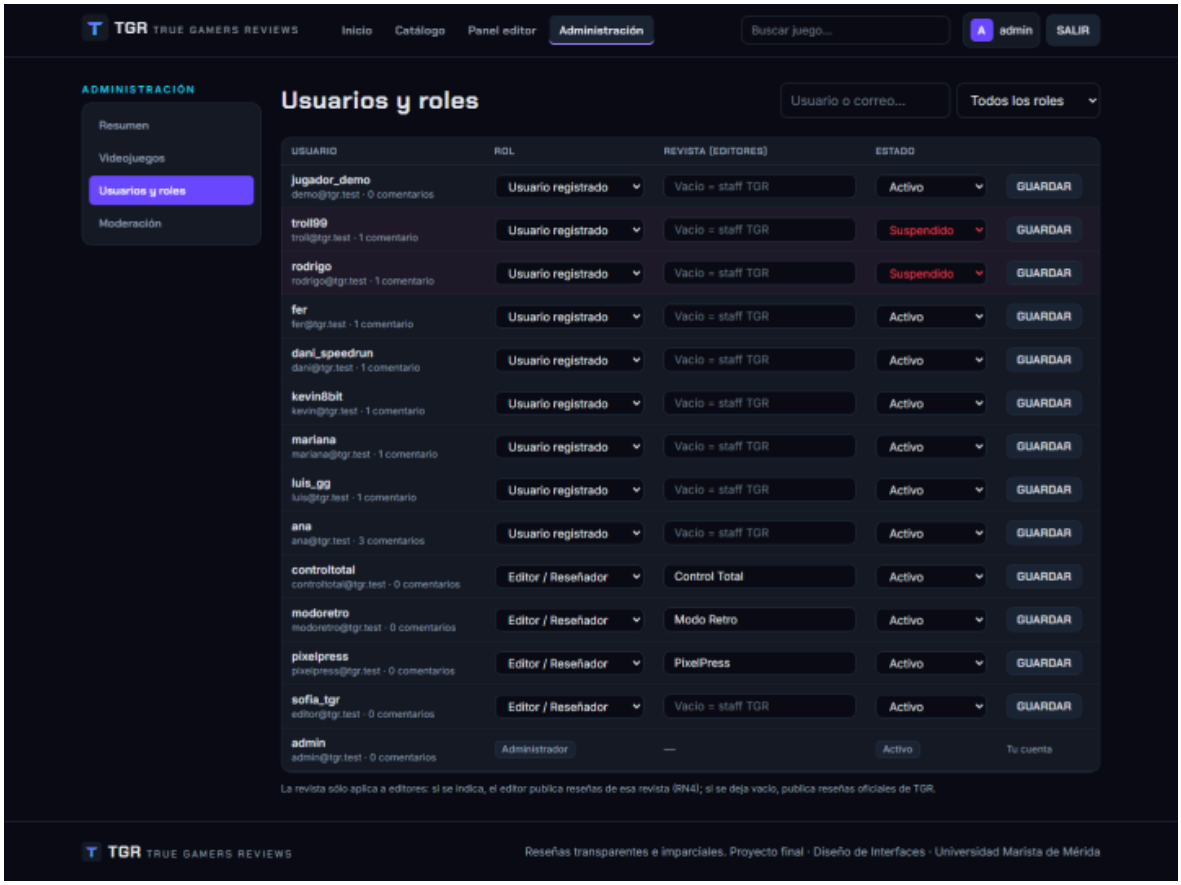


Figura 12. Administración de usuarios y roles.

Sistema de diseño

Concepto: “gamer, pero profesional”. El público objetivo son jugadores que pasan sesiones largas frente a la pantalla, por lo que se eligió un tema oscuro que reduce la fatiga visual y se asocia con la estética de los videojuegos. Los acentos violeta y cian dan el toque neón sin saturar, y la estructura limpia (tarjetas, espacio en blanco y jerarquía tipográfica clara) mantiene la seriedad de un medio editorial confiable. Cada una de las tres calificaciones tiene su propio color (TGR violeta #A48BFF, Revistas cian #22D3EE, Comunidad ámbar #F5B83D), pero siempre va acompañada de una etiqueta de texto y un ícono para cumplir RNF4. Otros colores del sistema: fondo base #0B0E14, línea de bordes #2A3347, primario suave para enlaces #A48BFF y acento para el foco de teclado #22D3EE. Todos los textos superan el contraste mínimo de 4.5:1 de WCAG 2.1 AA.

Color

Nombre	Uso principal	HEX	Muestra
Primario	Botones principales y elementos activos (5.9:1 con texto blanco)	`#6C47FF`	
Secundario (superficie)	Fondos de tarjetas y paneles	`#141925`	
Éxito	Confirmaciones, estados correctos y pros	`#22C55E`	
Error	Alertas, mensajes de error, contras y eliminar	`#F43F5E`	
Texto principal	Cuerpo de texto (contraste 15.8:1)	`#E6E9F2`	
Texto secundario	Subtítulos, ayudas y metadatos (contraste 7.4:1)	`#9AA3B8`	
Primario suave	Enlaces y acentos; calificación oficial TGR (contraste 7.0:1)	`#A48BFF`	
Acento	Foco de teclado y etiquetas; calificación de revistas (11.2:1)	`#22D3EE`	
Comunidad	Calificación de la comunidad y avisos (10.6:1)	`#F5B83D`	

Fondo base	Fondo de toda la aplicación (tema oscuro)	#0B0E14	
Línea	Bordes y divisores	#2A3347	

Tipografía (tipo de letra)

Tipografías: Google Fonts – Chakra Petch (títulos, etiquetas y calificaciones) e Inter (cuerpo de texto). Chakra Petch tiene terminaciones angulares que evocan interfaces de videojuegos; Inter está diseñada para pantallas y se lee bien en reseñas largas.
https://fonts.google.com/specimen/Chakra+Petch
https://fonts.google.com/specimen/Inter
• Títulos: Chakra Petch Bold, 48 px (H1) y 30 px (H2)
• Subtítulos: Chakra Petch SemiBold, 20 px
• Etiquetas: Chakra Petch SemiBold, 13 px, mayúsculas, espaciado 8 %
• Calificaciones: Chakra Petch Bold, 40 px
• Cuerpo: Inter Regular, 16 px, interlineado 160 %
• Texto pequeño: Inter Regular 14 px; leyendas: Inter Medium 12 px

Reglas de uso

1. Usar siempre los colores definidos (variables en Figma y tokens @theme en el código); nunca valores sueltos.
2. Mantener consistencia tipográfica: Chakra Petch sólo para títulos, etiquetas y números; Inter para lectura.
3. Cuidar los estados hover, focus (contorno cian de 2 px) y disabled (50 % de opacidad) en todos los controles.
4. Priorizar accesibilidad (WCAG 2.1 AA): contraste mínimo de 4.5:1, etiquetas visibles en formularios, texto alternativo en imágenes y enlace “Saltar al contenido”.
5. Las tres calificaciones nunca se distinguen sólo por color: siempre llevan etiqueta (TGR, Revistas, Comunidad) e ícono.
6. Un solo botón primario por vista; acciones secundarias en estilo ghost; acciones destructivas en estilo danger y con confirmación.
7. El buscador está en el inicio y en la barra superior para llegar a cualquier reseña en máximo 3 clics (RNF6).

Componentes

Las pantallas se arman con 8 componentes reutilizables de Figma, enlazados a las variables de color y a los estilos de texto del sistema de diseño.

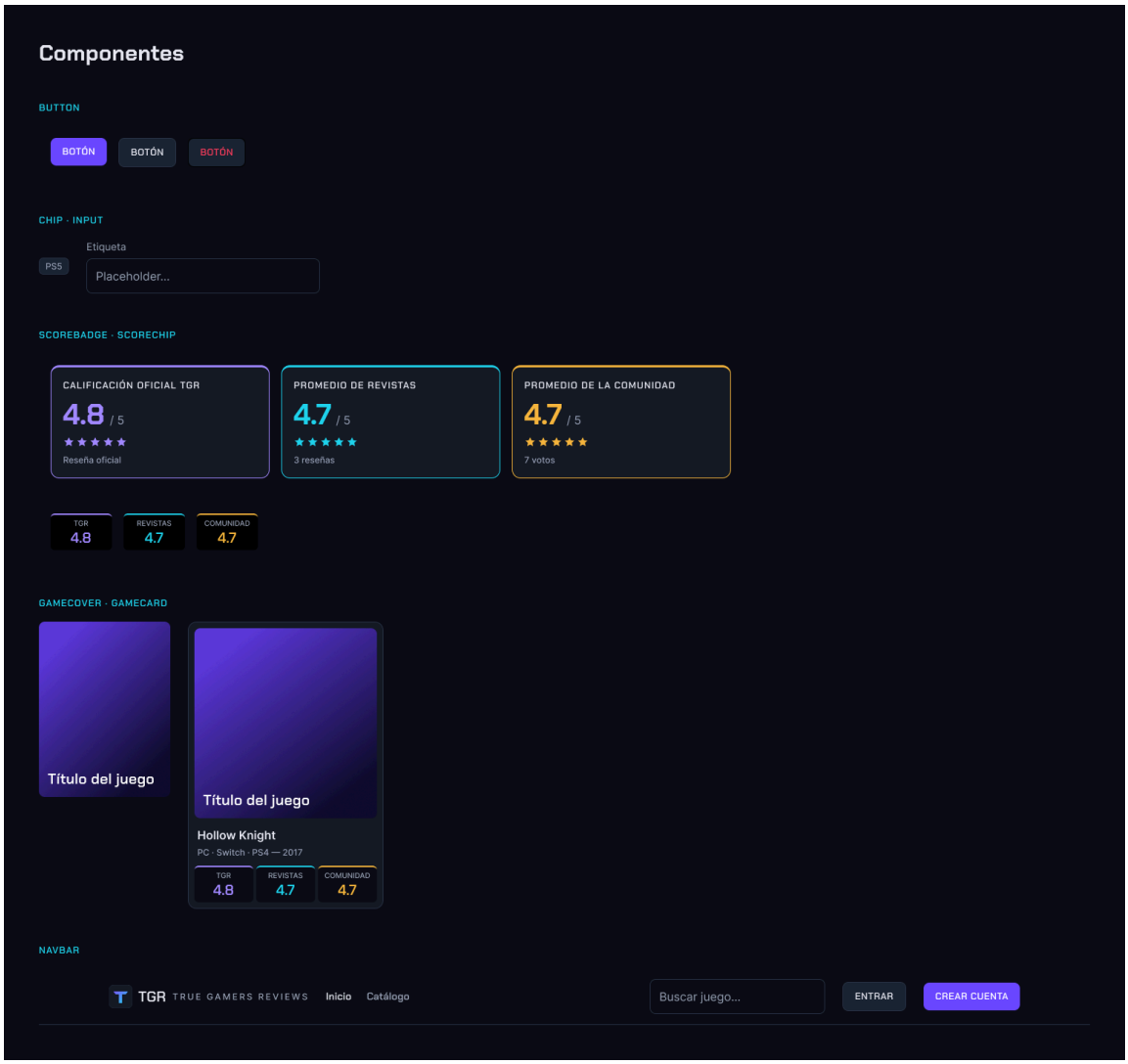


Figura 13. Componentes en Figma: Button (primary, ghost y danger), Chip, Input, ScoreBadge, ScoreChip, GameCover, GameCard y NavBar.

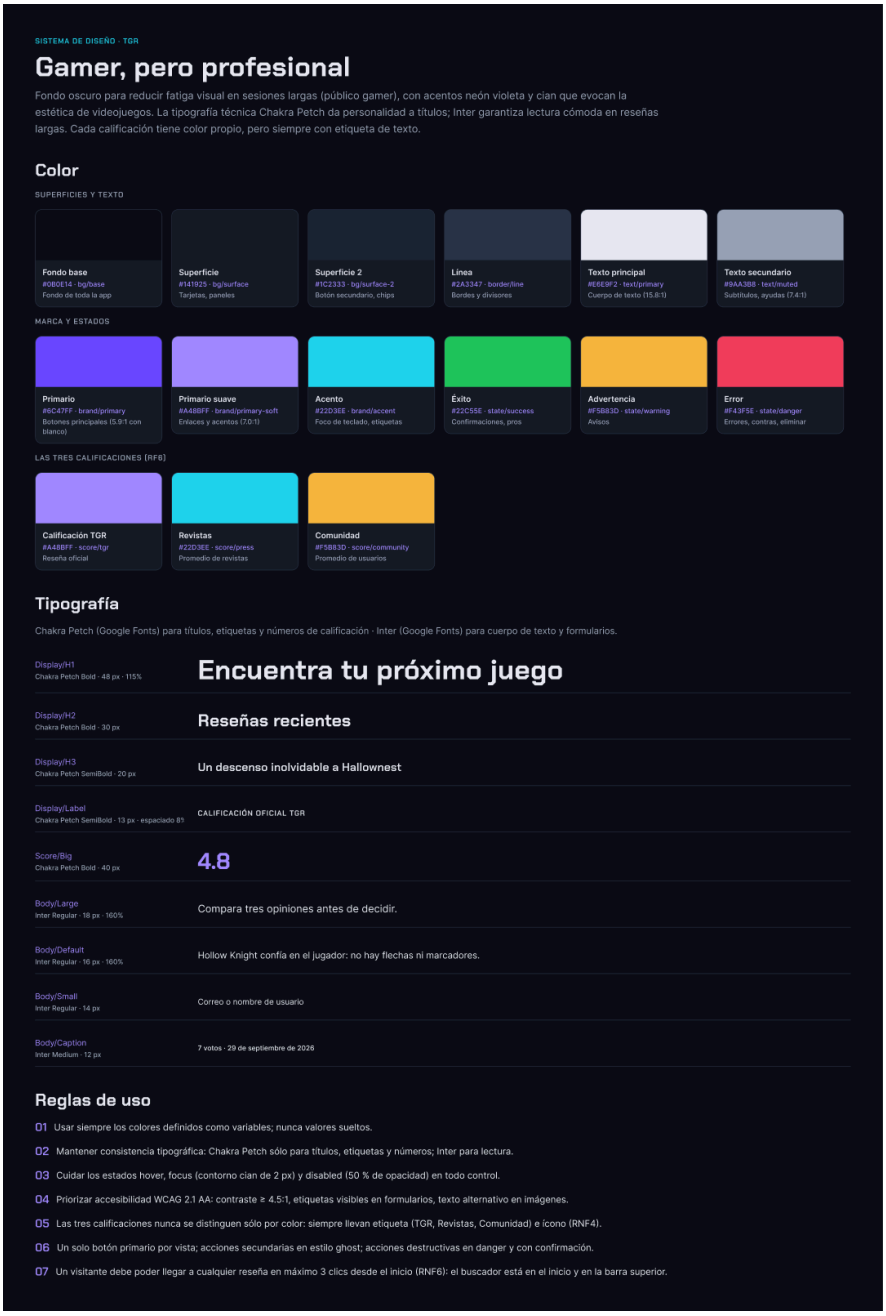


Figura 14. Fundamentos del sistema de diseño en Figma: color, tipografía y reglas de uso.